

1. (3 Punkte) Beim Backtracking für das Haus vom Nikolaus wird irgendwann `back([0, 3, 4, 1, 2, 3])` aufgerufen. Was sind die Argumente für die 5 folgenden `back`-Aufrufe?

```
2
1 3
0 4
```

Lösung:

```
[0, 3, 4, 1, 2, 3, 1]
[0, 3, 4, 1, 2, 3, 1, 0]
[0, 3, 4, 1, 2, 3, 1, 0, 4]
[0, 3, 4, 1, 3]
[0, 3, 4, 1, 3, 2]
```

2. (3 Punkte) Beim Backtracking für das 7-Damen-Problem wird irgendwann `back([0, 2, 4, 1, 3])` aufgerufen. Was sind die Argumente für die 6 folgenden `back`-Aufrufe?

Lösung:

```
[0, 2, 4, 6]
[0, 2, 4, 6, 1]
[0, 2, 4, 6, 1, 3]
[0, 2, 4, 6, 1, 3, 5]
[0, 2, 4, 6, 3]
[0, 2, 5]
```

3. (3 Punkte) Die Klasse `Fahrzeug` hat die Attribute `marke` und `baujahr`. Implementiere die Klasse `Fahrzeug`. Folgender Dialog soll möglich sein:

```
f = Fahrzeug("BMW", 2020)
print(f)
```

```
Das Fahrzeug BMW wurde im Jahr 2020 gebaut.
```

Lösung:

```
class Fahrzeug:
    def __init__(self, marke, baujahr):
        self.marke = marke
        self.baujahr = baujahr

    def __str__(self):
        return f"Das Fahrzeug {self.marke} wurde im Jahr {self.baujahr} gebaut."
```

4. (4 Punkte) Die Klasse `Tier` hat ein Attribut `name`. Dem Konstruktor von `Tier` wird der Wert für das Attribut `name` übergeben. Die Klasse `Hund` erbt von `Tier` und erhält ein weiteres Attribut `rasse`. Dem Konstruktor von `Hund` werden Werte für beide Attribute übergeben, der Konstruktor von `Hund` nutzt den Konstruktor von `Tier`. Implementiere die Klasse `Hund`.

Lösung:

```
class Hund(Tier):
    def __init__(self, name, rasse):
        super().__init__(name)
        self.rasse = rasse
```

5. (3 Punkte) Die Klasse `Spieler` hat das Attribut `punktzahl`. Beim Erstellen eines Spielers wird der Wert von `punktzahl` auf 0 gesetzt. Implementiere die Klasse `Spieler` unter Wahrung des Geheimnisprinzips mit getter und setter-Methode.

Lösung:

```
class Spieler:
    def __init__(self):
        self.__punktzahl = 0

    def setPunktzahl(self, x):
        self.__punktzahl = x

    def getPunktzahl(self):
        return self.__punktzahl
```

6. (6 Punkte) a. Implementiere eine Klasse `Film` mit den Attributen `regisseur` und `titel`.
b. Erzeuge zwei Film-Objekte `f1` und `f2`: *Inception* von Christopher Nolan und *Pulp Fiction* von Quentin Tarantino.
c. Erstelle eine Liste `filme` die `f1` und `f2` enthält.
d. Erstelle ein Dictionary `filmmap` mit dem Titel eines Films als key und dem Film-Objekt als value. `filmmap` soll zu Beginn Einträge für `f1` und `f2` enthalten.
e. Erweitere `filmmap` um einen Eintrag: *The Matrix* von den Wachowskis.
f. In `filmmap` seien noch mehrere Filme eingefügt worden. Welcher Ausdruck mit `filmmap` liefert den Regisseur von *Titanic* (wir nehmen an, dass der entsprechende Film in `filmmap` vorhanden ist).

Lösung:

```
class Film:
    def __init__(self, regisseur, titel):
        self.regisseur = regisseur
        self.titel = titel

f1 = Film("Christopher Nolan", "Inception")
f2 = Film("Quentin Tarantino", "Pulp Fiction")
filme = [f1, f2]
filmmap = {"Inception": f1, "Pulp Fiction": f2}
filmmap["The Matrix"] = Film("Wachowskis", "The Matrix")
filmmap["Titanic"].regisseur
```

7. (3 Punkte) Schreibe Code, der den folgenden Dialog möglich macht:

```
p1 = P(5)
p2 = P(8)
print(p1.a, p1.b)
print(p2.a, p2.b)
```

```
5 42
8 42
```

Lösung:

```
class P:
    def __init__(self, a):
        self.a = a
        self.b = 42
```

8. (5 Punkte) a. Gib für die beiden Klassen ein passendes Klassendiagramm an.

b. Was erscheint auf der Konsole?

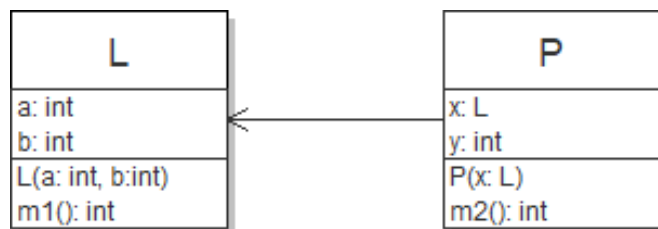
```
class L:
    def __init__(self,a,b):
        self.a = a
        self.b = b

    def m1(self):
        return self.a + self.b

class P:
    def __init__(self,x):
        self.x = x
        self.y = 3
    def m2(self):
        return self.x.m1() + self.y
```

```
k = L(4,5)
p = P(k)
print(p.m2())
```

Lösung:



b. 12

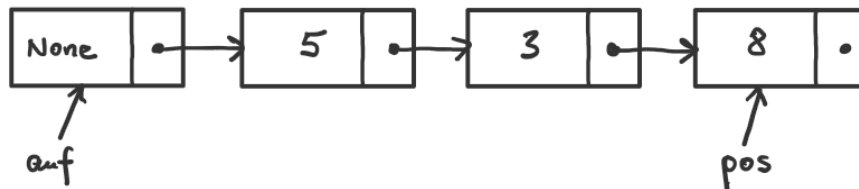
9. (2 Punkte) Liste ist unser selbstgebastelter Datentyp. Wir nummerieren die Einträge der Liste mit Zeiger **anf** an der Stelle 0.

- Welche Zahlenfolge findet sich in den Listenlementen?
- Gib den Index des Elements an, auf das der Zeiger **pos** zeigt.

```
a = Liste()
a.insert(1)
a.insert(2)
a.reset()
a.advance()
a.insert(3)
a.advance()
a.delete()
a.insert(4)
a.reset()
a.insert(6)
```

Lösung: a. 6 2 3 4 b. 0

10. (4 Punkte) Liste ist unser selbstgebastelter Datentyp. Das Bild zeigt die Situation der Liste a.



a. Was liefern die folgenden Anweisungen:

```
print(a.endpos())
print(a.elem())
```

b. Es werden folgende Anweisungen ausgeführt

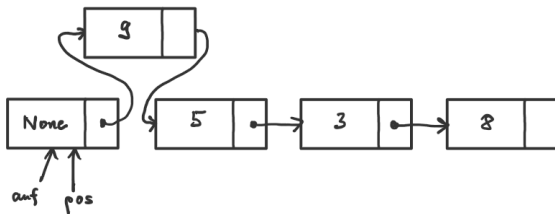
```
print(a.reset())
print(a.insert(9))
```

Zeichne die entstandene Situation.

c. Was liefert jetzt die Anweisung `a.elem()`

Lösung: a. True, ein Fehler wird geworfen

b.



c. 9

11. (3 Punkte) Welche Ausgabe erscheint auf der Konsole?

```
class VerweisBox:
    def __init__(self, inhalt, unten=None, oben=None):
        self.inhalt = inhalt
        self.unten = unten
        self.oben = oben
        if unten is not None: self.unten.oben = self
        if oben is not None: self.oben.unten = self

    def __str__(self):
        return self.inhalt

a = VerweisBox('a')
b = VerweisBox('b')
c = VerweisBox('c',b,a)
d = VerweisBox('d',a,c)
print(b.oben.oben)
print(a.unten.unten)
print(d.oben.unten.unten)
```

Lösung: a, d, a

12. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
b. Was erscheint auf der Konsole?

```
a = []

for i in range(4):
    a.append(i)

print(a.pop(), end = ' ')
print(a.pop(), end = ' ')

for i in range(5,8):
    a.append(i)

while a:
    print(a.pop(), end = ' ')
```

Lösung: a. Keller b. 3 2 7 6 5 1 0

13. (2 Punkte) a. Welche Datenstruktur wird hier verwendet (Liste/Keller/Schlange)?
b. Was erscheint auf der Konsole?

```
from collections import deque
a = deque([])

for i in range(4):
    a.append(i)

print(a.popleft(), end = ' ')
print(a.popleft(), end = ' ')

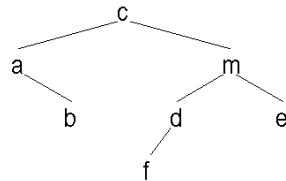
for i in range(5,8):
    a.append(i)

while a:
    print(a.popleft(), end = ' ')
```

Lösung: a. Schlange b. 0 1 2 3 5 6 7

14. (4 Punkte) a. Zeichne den Baum.
b. Traversiere den Baum in preorder, inorder und postorder

```
..e
.m
..d
...f
c
..b
.a
```

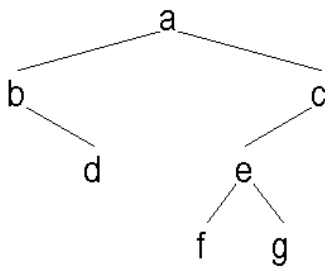


Lösung:

```
preorder: c a b m d f e
inorder: a b c f d m e
postorder: b a f d e m c
```

15. (3 Punkte) Schreibe eine Funktion `baum`, die den abgebildeten Baum zurückgibt. Nutze dazu unsere Node-Klasse. Verschachte die Node-Aufrufe aus Lesbarkeitsgründen höchstens einmal und gib den temporären Teilbäumen sprechende Namen.

```
class Node:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right
```



Lösung:

```
def baum():
    e = Node('e', Node('f'), Node('g'))
    c = Node('c', e, None)
    b = Node('b', None, Node('d'))
    a = Node('a', b, c)
    return a
```

Aufgabe:	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	Summe:
Punkte:	3	3	3	4	3	6	3	5	2	4	3	2	2	4	3	50